NTNU

Norwegian University of
Science and Technology

Practical Anonymous Communication with Homomorphic Encryption

Angelo, Imre

Submission date: July 2026
Main supervisor: Associate Professor Park, Jeongeun, NTNU

Norwegian University of Science and Technology
Department of Information Security and Communication Technology

Problem description

Many anonymous communication systems have been proposed and implemented with the goal of allowing users to exchange messages over a network without revealing who is communicating with whom. Most of these designs rely on the *threshold model* to provide anonymity, wherein some components of their infrastructure must be behaving honestly. Systems with stronger anonymity guarantees generally suffer from poor scalability or are impractical to instantiate.

This thesis examines a new construction called a (Somewhat) Practical Anonymous Router (sPAR), which addresses the trust and practicality concerns of existing systems. The construction relies only on well-established Fully Homomorphic Encryption (FHE) schemes, making it realistically implementable. While sPAR presents a promising new avenue for constructing anonymous communication systems, it remains a theoretical construction. A concrete implementation and a deeper investigation is therefore needed to evaluate its practical performance and viability in larger networks.

In an effort to investigate sPAR's viability, this thesis aims to implement the scheme and conduct a structured evaluation of its performance with respect to the number of participants in the system.

Research Questions:

- What is the computational cost of sPAR, and how does it compare to the theoretical complexity described in the original paper?
- How does the performance of sPAR scale with the number of participants, and at what point does it become impractical?

Approved: 2026-04-01 – Associate Professor Park, Jeongeun, NTNU (Main supervisor)

Abstract

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

This is the second paragraph. Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

And after the second paragraph follows the third paragraph. Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

Sammendrag

After this fourth paragraph, we start a new paragraph sequence. Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

This is the second paragraph. Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

Preface

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

Contents

List of Figures

List of Tables

List of Algorithms

Chapter 1
Background

Anonymous communication aims to enable users to exchange messages over a network without revealing who is communicating with whom, thereby protecting both sender and receiver identities. Many such systems have been proposed and implemented over the years, such as Tor, I2P, and mixnets. Most of these designs depend on the *threshold model* to provide anonymity, requiring at least some components of their infrastructure to behave honestly. In practice, this assumption is increasingly strained by large-scale surveillance, correlation attacks, and the growing centralization of internet infrastructure [SEV+15].

Systems with stronger anonymity guarantees based on Multi-Party Communication (MPC) exist. However, these suffer from poor scalability due to their high interactivity, making them impractical for large-scale networks. A Dining Cryptographers network (DC-net), for example, can provide anonymity even if there are no trusted components in the network infrastructure. Doing so requires every participant to interact with every other participant, for every round of messages sent.

Recently, a new line of research has proposed fundamentally different designs, aiming to offer strong anonymity even in the presence of compromised or adversarial components in the network actively attempting to break anonymity. The first such design was the Non-Interactive Anonymous Router (NIAR) [SW21], which proposing using Multi-Client Functional Encryption (MCFE) to allow a single, untrusted router to shuffle or permute messages from a set of clients *obliviously*; without learning anything about the permutation. Thereby making it infeasible for the router to link any of the messages to its sender.

NIAR was a breakthrough in anonymous communication, not only showing the theoretical possibility of designing anonymous routers, but also that such routers can be *non-interactive*: clients only send one message to the router and do not need to perform any additional interaction to achieve anonymity. Using a centralized (anonymous) router in this way yields similar benefits to existing MPC-based systems,

without needing any interactions between clients. However, there are a number of problems that makes NIAR impractical, if not impossible, to implement in practice.

1. The cryptographic components required by NIAR are extremely computationally expensive or not known to be implementable in practice.
2. Achieving anonymity for recipients assumes the existence of a very strong primitive called *indistinguishability obfuscation with sub-exponential security*: guaranteeing that no efficient adversary can distinguish between the obfuscations of two functionally equivalent circuits with more than a sub-exponentially small advantage. The practicality of such a construction is unknown.
3. Reusing the same setup across multiple rounds makes messages from the same sender linkable, unless the expensive setup phase is rerun for every round.

These problems were highlighted in sPAR

- Anonymous communication
- Current solutions
- Problems with current solutions
- FHE as a new approach
- FHE 'holy grail' of privacy-preserving communication
- sPAR
- RSA is multiplicatively homomorphic, but (follow-up) paper theorized a *fully* homomorphic scheme may be possible [Gen09; RAD78]
- *Contributions*
  - Port of TFHE-style external/internal products to BGV-RNS
  - A reusable OpenFHE library
  - Complete sPAR implementation
  - First empirical evaluation of it

Structure of the thesis

- Building blocks and related works; FHE, gadget decomposition, TFHE, etc.
- Explanation of the code
- Experimental setup i.e. how the numbers where found
- Results, benchmarks, noise analysis
- Future work

Chapter 2
Preliminaries

This section details the fundamental building blocks of the sPAR protocol, and the protocol itself. To achieve a reasonable depth the protocol utilizes the *external product*, a primitive from TFHE bootstrapping that allows homomorphic multiplication of binary digits with near-constant noise bounds — enabling a large number of binary multiplication without consuming a level/bootstrapping. This primitive was ported to BGV-RNS for this thesis, which does not natively allow digit decomposition as normally used by the external product.

2.1 Fully Homomorphic Encryption

Homomorphic Encryption (HE) allows computation directly on encrypted data: an untrusted party can evaluate a function over ciphertexts without learning anything about the underlying plaintexts. Schemes are usually distinguished by the circuits they can evaluate — *leveled* schemes support circuits up to a fixed multiplicative depth, whereas true FHE supports arbitrary depth. The constructions in this thesis are leveled, so the additional machinery required to lift a leveled scheme to a fully homomorphic one is not needed here.

With few exceptions, the best-performing FHE frameworks [**BFV**; **CKKS**; **TFHE**; BGV11] base their security on the Learning With Errors (LWE) problem introduced by Regev [**regev:2009**], or on one of its ring variants. Throughout, $\|\cdot\|$ denotes the infinity norm $\|\cdot\|_\infty$, that is, the largest absolute value among the coefficients of a polynomial (equivalently, among its residues in the representation of **??**).

Definition 2.1. Working ring All plaintexts and ciphertexts are elements of the cyclotomic ring $R_Q = \mathbb{Z}_Q[X]/(X^N + 1)$, where N is a power of two and Q is the ciphertext modulus. Elements are polynomials of degree at most $N - 1$ whose coefficients are reduced into a centered range about zero.

Definition 2.2. Centered residue representation For a modulus q, an element of $\mathbb{Z}_q$ is identified with its balanced representative through the map

$$[a]_q = \big(a + \lfloor q/2 \rfloor \bmod q\big) - \lfloor q/2 \rfloor. \tag{2.1}$$

The flooring makes the map well defined for every q, and its image is the run of q consecutive integers centered at the origin, so that $|[a]_q| \le \lfloor q/2 \rfloor$. Working with balanced representatives is what allows every noise term to be bounded by $\|\epsilon\| \le \lfloor q/2 \rfloor$ rather than by q.

Definition 2.3. RLWE ciphertext An Ring Learning With Errors (RLWE) ciphertext is a pair $\overrightarrow{\mathbf{x}} = (b, a) \in R_Q^2$ consisting of a uniformly random **public element** a and a **masked element**

$$b = a \cdot s + \Delta m + \epsilon, \tag{2.2}$$

where s is a persistent secret key, $m \in R_t$ is the plaintext for a plaintext modulus t, Δ is the message scaling factor, and ϵ is a freshly sampled noise term. These are drawn as

$$s \xleftarrow{\$} \{-1, 0, 1\}^N, \qquad \epsilon \xleftarrow{\$} \chi, \qquad a \xleftarrow{\$} R_Q, \tag{2.3}$$

with s a ternary secret and χ a discrete Gaussian error distribution. We write $\mathsf{RLWE}_s(m)$ for an encryption of m under s, abbreviated to $\mathsf{RLWE}(m)$ when the key is clear from context.

Definition 2.4. **Public key** A public key is an encryption of zero, $pk = \mathsf{RLWE}_s(0) = (a \cdot s + \epsilon, a)$. Because encryptions add homomorphically (below), it enables encryption without the secret key: one simply adds the encoded message onto pk, i.e. $\textsc{Encrypt}(m, pk) = pk + (\textsc{Encode}(m), 0)$.

Definition 2.5. **Encryption** $\textsc{Encrypt}(m) \mapsto \mathsf{RLWE}(m)$ returns a fresh ciphertext encrypting m; given a public key pk, the map $\textsc{Encrypt}(m, pk) \mapsto \mathsf{RLWE}(m)$ does the same without the secret key.

Definition 2.6. **Decryption** $\textsc{Decrypt}(\mathsf{RLWE}(m), s) \mapsto m$ recovers the plaintext using the secret key s. Recovery is correct only while the ciphertext noise stays within a scheme-dependent bound; the concrete bound for the scheme used in this thesis is given in Theorem 2.10.

FHE schemes support homomorphic addition and multiplication, and typically one of the two comes for *free*, in the sense that applying it directly to the ciphertexts applies it to the underlying plaintexts with no additional machinery. Schemes built on RLWE have free addition:

$$\begin{aligned}\vec{\mathbf{x}} + \vec{\mathbf{y}} &= (b_x, a_x) + (b_y, a_y) \\ &= (a_x s + \Delta m_x + \epsilon_x,\ a_x) + (a_y s + \Delta m_y + \epsilon_y,\ a_y) \\ &= \big((a_x + a_y)s + \Delta(m_x + m_y) + (\epsilon_x + \epsilon_y),\ a_x + a_y\big).\end{aligned}$$

Setting $a = a_x + a_y$ and $\epsilon = \epsilon_x + \epsilon_y$ gives $\vec{\mathbf{x}} + \vec{\mathbf{y}} = (as + \Delta(m_x + m_y) + \epsilon,\ a) = \mathsf{RLWE}_s(m_x + m_y)$.

Definition 2.7. **RLWE Addition** Following ... define addition as...

$$\mathsf{RLWE}_s(m_x) + \mathsf{RLWE}_s(m_y) = \mathsf{RLWE}_s(m_x + m_y). \tag{2.4}$$

The sum encrypts $m_x + m_y$ but carries the combined noise $\epsilon_x + \epsilon_y$, and it is precisely this accumulation that shapes the design of every LWE-based scheme. Grounding security in LWE carries an unavoidable cost: each ciphertext must hold a random error term. This is inconsequential for ordinary encryption, but in the homomorphic setting the error is the resource that determines how much computation a ciphertext can withstand. Every operation enlarges it, and a ciphertext decrypts correctly only while its error remains beneath a scheme-dependent threshold; once that budget is spent, the plaintext is irrecoverable.

Two broad remedies exist. The first is *bootstrapping*, due to Gentry [Gen09]: the decryption function is itself evaluated homomorphically on an encryption of the ciphertext, yielding a fresh encryption of the same plaintext in which the accumulated

error has been discharged and replaced by the comparatively small error of the bootstrapping procedure alone. Bootstrapping removes any restriction on circuit depth, but it is expensive; this thesis avoids it entirely. The constructions considered here are leveled and instead rely on the external product (**??**), whose noise growth is mild enough that the full protocol completes within a single budget. The second remedy, natural to leveled schemes, is to spend the budget sparingly: both by shedding accumulated error through modulus switching, and by restraining its growth at the source. The standard instrument for the latter is the *gadget decomposition* [**BV11**], treated in detail in **??**, which is also the foundation of the external product on which the whole construction rests. Homomorphic multiplication, by contrast, is considerably more involved and noisy in LWE-based schemes; the present work sidesteps native ciphertext-ciphertext multiplication altogether, obtaining multiplication of binary digits from external products instead.

Definition 2.8. Modulus switching Modulus switching transforms a ciphertext $\overrightarrow{\mathbf{x}} \in R_Q^2$ under a modulus Q into a ciphertext $\overrightarrow{\mathbf{x}}' \in R_{Q'}^2$ under a smaller modulus $Q' < Q$ that decrypts to the same plaintext, by scaling each component by Q'/Q and rounding to the nearest integer. As the scaling reduces the noise by the same factor, it is the primary noise-management tool in leveled schemes, allowing continued computation without the cost of bootstrapping. Its realization in the residue setting is deferred to **??**.

Definition 2.9. Key-switching Key-switching converts a ciphertext encrypted under one secret key into a ciphertext of the same plaintext under a different secret key. The techniques it relies on are developed in **??**.

2.1.1 Multi-Party FHE

Anonymous routing places the ciphertexts in the hands of a server that is not merely untrusted but may actively attempt to break privacy, so no single party can be allowed to hold the decryption key. Several multi-party variants of FHE address this; the method used in this thesis distributes both key generation and decryption across the participants. Each party i samples its own secret key s_i and publishes a corresponding public share, and the shares are aggregated once during setup into a single joint public key under which everyone encrypts. The associated joint secret $s = \sum_i s_i$ is never reconstructed by any party.

Decryption proceeds in two stages. Each party partially decrypts a ciphertext with its own share s_i, adding extra noise to the partial result — a step known as *noise flooding* — so that the released value leaks nothing about s_i. Because decryption is linear in the secret, the server then combines the partial decryptions into the plaintext without any party ever learning the joint secret.

2.1.2 The BGV scheme

While most of the development in this thesis is scheme-agnostic, the implementation is built on the BGV scheme [BGV11]. BGV is a convenient choice for several reasons: it places the message in the *least* significant bits of the ciphertext, so that a binary value encodes cleanly as $v \in \{0,1\} \mapsto m \in \{0,1\}$ — exactly the behaviour the digit-wise external product depends on; it supports SIMD batching natively (below); it has a mature residue-based instantiation in OpenFHE [BAB+22]; and its theory is close enough to BFV that support for the latter would require little additional work.

Concretely, BGV fixes the message scale to $\Delta = 1$ and scales the noise by the plaintext modulus, $\epsilon = t \cdot \epsilon_{\mathsf{rlwe}}$, so that the masked element of Theorem 2.3 reads $b = a \cdot s + m + t\,\epsilon_{\mathsf{rlwe}}$ for a base noise term $\epsilon_{\mathsf{rlwe}} \xleftarrow{\$} \chi$.[1] To keep the exposition general, ciphertexts are written throughout in the form of Theorem 2.3, with the BGV specialization left implicit.

Definition 2.10. BGV decryption

$$\textsc{Decrypt}(\overrightarrow{\mathbf{x}}, s) = \big[\, [b - a \cdot s]_Q \,\big]_t, \tag{2.5}$$

which is valid as long as $\|\epsilon\| < Q/2$. Under this bound the inner centered reduction recovers $m + t\,\epsilon_{\mathsf{rlwe}}$ exactly over the integers, and the outer reduction modulo t then returns m.

Definition 2.11. SIMD batching If the plaintext modulus t is a prime satisfying $t \equiv 1 \pmod{2N}$, then BGV admits per-slot SIMD packing [SV11]. Batching reinterprets a plaintext, ordinarily a polynomial of degree at most $N-1$, as N independent constants $[c_i]_t$, and homomorphic operations then act slot-wise; for example $(1,2)\cdot(2,3) = (2,6)$, in contrast to the polynomial product $(1+2X)(2+3X) = (2,7,6)$.

[1] Some noise-management strategies, such as OpenFHE's `FLEXIBLEAUTO`, vary Δ in practice.

2.2 Residue Number Systems

In BGV, BFV and CKKS the ciphertext modulus Q is large, much larger than a computer word/register (typically 64 bits on modern computers). Numbers that do not fit inside a register must be handled with multi-precision arithmetic, which is known to be slow due to its serial nature that does not allow parallelization. High-performance instantiations avoid this problem by using a Residue Number System (RNS) where large numbers are represented as a set of smaller, independent residues via the Chinese Remainder Theorem (CRT). RNS variants of BGV, BFV and CKKS are the standard in practice; OpenFHE, for example, only supports the RNS versions of these schemes [BAB+22].

Definition 2.12. Coefficient Form As polynomials have a well-known/predictable structure, we can omit the redundant X variable and represent $a \in R_Q$ uniquely by its coefficients. In this thesis I will consider these forms functionally equivalent, and refer to both (**??**) and (2.6) as *canonical*.

$$a = (c_0, c_1, \ldots, c_{N-1}) \tag{2.6}$$

Let $Q = q_0 \ldots q_{k-1}$ with $\gcd(q_i, q_j) = 1 \; \forall \; i \neq j$, so that CRT allows decomposition into smaller residues. If $\max_i\{q_i\}$ fits in a computer word then we can use CRT on the coefficients of a polynomial to work exclusively with integers that fit inside a computer word.

Definition 2.13. First RNS Form (CRT Form) Given a polynomial $a \in R_Q$ where $Q = q_0 q_1 \ldots q_{k-1}$ is chosen as a product of k small primes (small meaning they each fit inside a computer register), each coefficient c_i is uniquely represented by its k residues via the Chinese Remainder Theorem. Then the **CRT form** of a (2.7) is a collection of smaller polynomials called *towers*, *residues* or *limbs*.

$$\begin{pmatrix} a \mod q_0 \\ a \mod q_1 \\ \vdots \\ a \mod q_{k-1} \end{pmatrix} = \begin{pmatrix} c_0 \bmod q_0 & c_1 \bmod q_0 & \ldots & c_{N-1} \bmod q_0 \\ c_0 \bmod q_1 & c_1 \bmod q_1 & \ldots & c_{N-1} \bmod q_1 \\ \vdots & \vdots & \ddots & \vdots \\ c_0 \bmod q_{k-1} & c_1 \bmod q_{k-1} & \ldots & c_{N-1} \bmod q_{k-1} \end{pmatrix} \tag{2.7}$$

Addition and subtraction of two polynomials in R_Q is equivalent to element-wise addition/subtraction of their CRT forms (modulo q_i). As each element fits within a computer word, no carries propagate between them, and all pairs of elements can be added or subtracted in parallel.

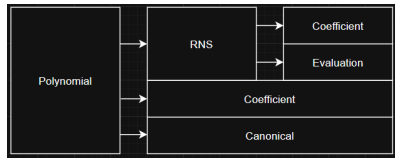
2.2.1 Number Theoretic Transform

The Number Theoretic Transform (NTT) converts a negacyclic convolution in the coefficient domain into point-wise multiplication in the NTT domain, and multiplication in R_{q_i} is exactly such a convolution of the coefficient vectors. Applying the NTT to each residue of the CRT form (2.7) therefore makes polynomial multiplication element-wise as well. The negacyclic NTT of size N requires each modulus to satisfy $q_i \equiv 1 \pmod{2N}$, which constrains the choice of RNS primes.

Definition 2.14. Second RNS form (Double CRT/NTT/Evaluation) By applying the NTT to the CRT form (2.7) we gain the ability to perform polynomial multiplication in parallel, without losing the ability to perform addition/subtraction in parallel.

$$\begin{pmatrix} \mathsf{NTT}(a \bmod q_0) \\ \mathsf{NTT}(a \bmod q_1) \\ \vdots \\ \mathsf{NTT}(a \bmod q_{k-1}) \end{pmatrix} = \begin{pmatrix} \hat{a}_{0,0} & \hat{a}_{0,1} & \dots & \hat{a}_{0,N-1} \\ \hat{a}_{1,0} & \hat{a}_{1,1} & \dots & \hat{a}_{1,N-1} \\ \vdots & \vdots & \ddots & \vdots \\ \hat{a}_{k-1,0} & \hat{a}_{k-1,1} & \dots & \hat{a}_{k-1,N-1} \end{pmatrix} \tag{2.8}$$

Remark 2.15. Rather confusingly, OpenFHE calls this form *Coefficient* format. Both RNS forms are represented by an `DCRTPoly` object, so a polynomial in CRT form is represented by a `DCRTPoly` object in `Format::Coefficient` mode.



The evaluation form (2.8) is the preferred form, as it supports parallel addition, subtraction and multiplication, but it still does not natively support *all* necessary operations. Because switching between canonical/vector, CRT and NTT mode is relatively expensive — requiring (inverse) CRTs and NTTs — much of the relevant literature [GHS12; BEHZ16; HPS18; KPZ21] improves upon existing functions by finding new approaches and approximations that limit the number of conversions required.

2.2.2 Base Conversion

The evaluation form (2.8) makes addition and multiplication cheap, but pins the modulus: the residues are independent and non-positional, so the magnitude of a value — and with it division and rounding, the operations noise management is made of — is inaccessible without CRT reconstruction. Changing the modulus fares no better, as a representation in $\mathcal{B}_Q$ says nothing about a value modulo anything outside $\mathcal{B}_Q$. This subsection recovers both.

The operations are best understood through the pattern they serve: the temporary modulus extension introduced for key switching (section 2.3) by Gentry–Halevi–Smart [GHS12]. A value is raised from Q to an extended modulus QP, multiplied there against operands carrying a factor P, and brought back down by a division by P: the payload returns to its original scale, while the noise of the intermediate computation shrinks by the same factor. One primitive and three derived operations realize the trip.

Operation	Computes	Error
FastBConv	$[x]_{\mathcal{B}_Q} \mapsto [x + \alpha_x Q]_{\mathcal{B}}$	exact for the lift $x + \alpha_x Q$
FastBaseExtend	$[x]_{\mathcal{B}_Q} \mapsto [x + \alpha_x Q]_{\mathcal{B}_{QP}}$	exact, as an extension
Scale	$[x]_{\mathcal{B}_Q} \mapsto [xP]_{\mathcal{B}_{QP}}$	exact
ApproxModDown	$[x]_{\mathcal{B}_{QP}} \mapsto [\lfloor x/P \rceil - \alpha]_{\mathcal{B}_Q}$	additive, small α

Definition 2.16. RNS Basis Given a modulus $Q = \prod_{i=0}^{k-1} q_i$ composed of k pairwise coprime factors, the *RNS basis* $\mathcal{B}_Q = \{q_0, q_1, \ldots, q_{k-1}\}$ is the set of moduli underlying the CRT form (2.7) of $\mathbb{Z}_Q$.

By a slight abuse of notation, the centered representation of Theorem 2.2 extends to bases: $[a]_{\mathcal{B}_Q} = ([a]_{q_0}, \ldots, [a]_{q_{k-1}})$ denotes the CRT form (2.7) of a, and $\mathsf{NTT}([a]_{\mathcal{B}_Q})$ its evaluation form (2.8).

Definition 2.17. Base Conversion Let $\mathcal{B}_Q = \{q_0, \ldots, q_{k-1}\}$ and $\mathcal{B} = \{b_0, \ldots, b_{k'-1}\}$ be RNS bases with $\gcd(Q, B) = 1$, where $B = \prod_j b_j$. Base conversion is the map that takes the representation of x in $\mathcal{B}_Q$ to the representation of $[x]_Q$ in $\mathcal{B}$, i.e.

$$\left([x]_{q_0}, \ldots, [x]_{q_{k-1}}\right) \longmapsto \left([x]_{b_0}, \ldots, [x]_{b_{k'-1}}\right)$$

When the residues in $\mathcal{B}$ are appended to those in $\mathcal{B}_Q$, so that x becomes represented in the *extended* basis $\mathcal{B}_Q \cup \mathcal{B}$ for the modulus QB, the operation is also referred to as a **base extension**.

Fast Base Conversion

The naive conversion reconstructs x from its residues through the inverse CRT and reduces the result against each modulus of the new basis — reintroducing exactly the multi-precision arithmetic that RNS exists to avoid. Fast base conversion [BEHZ16] stays on the residues. Writing $\hat{q}_i = Q/q_i$ for the CRT factors of $\mathcal{B}_Q$, $[\hat{q}_i^{-1}]_{q_i}$ for their modular inverses and $x_i = [x]_{q_i}$ for the residues of the operand,

$$\textsc{FastBConv}(x, \mathcal{B}_Q, \mathcal{B}) = \left\{ \left[\sum_{i=0}^{k-1} \left[x_i \, \hat{q}_i^{-1} \right]_{q_i} \cdot \hat{q}_i \right]_b \right\}_{b \in \mathcal{B}} \tag{2.9}$$

The sum in (2.9) is never reduced modulo Q; it evaluates to the integer

$$\tilde{x} \;=\; \sum_{i=0}^{k-1} \left[x_i \, \hat{q}_i^{-1} \right]_{q_i} \cdot \hat{q}_i \;=\; [x]_Q + \alpha_x Q, \qquad |\tilde{x}| \le \tfrac{k}{2}\, Q, \tag{2.10}$$

since $\tilde{x} \equiv x \pmod{q_i}$ for every $q_i \in \mathcal{B}_Q$ while each centered digit is bounded by $q_i/2$. FastBConv therefore computes the $\mathcal{B}$-residues of the *lift* $\tilde{x}$ exactly, and is approximate only as a conversion of x: the output represents $[x]_Q + \alpha_x Q$ for an unknown overflow $\alpha_x \in \mathbb{Z}$ with $|\alpha_x| \le \lceil k/2 \rceil$. Recovering α_x, and with it the exact conversion, is possible at additional cost [HPS18]; none of the operations built on the primitive requires it.

The conversion is *fast* under one standing assumption and one caveat. The assumption: the constants $[\hat{q}_i]_b$ and $[\hat{q}_i^{-1}]_{q_i}$ depend only on the two bases, and are precomputed once and stored (section 3.1); what remains at runtime is $k \cdot k'$ single-word multiply-accumulates per coefficient. The caveat: (2.9) operates on the CRT form, so an operand held in evaluation form must first be brought back with the inverse NTT — in practice the dominant cost of a conversion.

Fast Base Extension

Let $\mathcal{B}_P = \{p_0, \ldots, p_{k'-1}\}$ be a second basis with every modulus coprime to those of $\mathcal{B}_Q$, and write $\mathcal{B}_{QP} = \mathcal{B}_Q \cup \mathcal{B}_P$ for the extended basis. Raising x from $\mathbb{Z}_Q$ to $\mathbb{Z}_{QP}$ requires only the new residues: the Q-limbs are kept unchanged, and the P-limbs — the limbs modulo the $p_j \in \mathcal{B}_P$ — are produced by $\textsc{FastBConv}(x, \mathcal{B}_Q, \mathcal{B}_P)$ (Algorithm 2.1).

Proposition 2.18. Exactness as an extension *The output of Algorithm 2.1 is the exact RNS representation in $\mathcal{B}_{QP}$ of the lifted integer $\tilde{x} = [x]_Q + \alpha_x Q \in \mathbb{Z}_{QP}$: the appended P-limbs are those of $\tilde{x}$ by* (2.10)*, and the reused Q-limbs agree with $\tilde{x}$ because $\tilde{x} \equiv x \pmod{q_i}$.*

Algorithm 2.1 Fast Base Extension

Input: $\mathsf{NTT}([a]_{\mathcal{B}_Q})$ and a basis $\mathcal{B}_P$
Output: $\mathsf{NTT}([a + \alpha_a Q]_{\mathcal{B}_{QP}})$ for some $\alpha_a \in \mathbb{Z}$, $|\alpha_a| \leq \lceil k/2 \rceil$
1: $a_Q \leftarrow \textsc{InverseNTT}(a)$
2: $\mathsf{conv} \leftarrow \sum_{i=0}^{k-1} [a_Q[i] \cdot \hat{q}_i^{-1}]_{q_i} \cdot \hat{q}_i$ ▷ Fast Base Conversion
3: **for** $p \in \mathcal{B}_P$ **do**
4: $\quad a \leftarrow a \cup \mathrm{NTT}([\mathsf{conv}]_p)$
5: **end for**
6: **return** a

The approximation of the primitive is thus not an error in any residue, but a change of representative: the extension represents, exactly, an integer that is congruent to x modulo Q and bounded by $\frac{k}{2}Q$.

Scale

The raising leg of the modulus extension needs operands that carry the factor P: whatever is meant to survive the final division must be P-scaled before it. I refer to the operation supplying them, "*extend and scale by* P", as $\textsc{Scale}(x, P) \mapsto [xP]_{\mathcal{B}_{QP}}$. Composing the extension with a multiplication defines it,

$$\textsc{Scale}(x, P) = \textsc{FastBaseExtend}(x, \mathcal{B}_Q, \mathcal{B}_P) \cdot P \tag{2.11}$$

and the composition is exact even though its ingredient is not, because multiples of Q are annihilated by P in $\mathbb{Z}_{QP}$:

$$(x + \alpha_x Q) \cdot P = xP + \alpha_x QP \equiv xP \pmod{QP}$$

In RNS the operation is far cheaper than (2.11) suggests: no conversion needs to take place at all. Every P-limb of xP is zero — each p_j divides P — so the limbs that the extension would have to compute are known in advance. Each Q-limb is multiplied by $[P]_{q_i}$ and k' zero limbs are appended (Algorithm 2.2); Scale never leaves the evaluation form, and is limb-local, NTT-free and exact.

Remark 2.19. Read right to left, (2.11) states that a multiplication by P collapses the two raising operations into one: P-scaling the output of Algorithm 2.1 yields the same element of $\mathbb{Z}_{QP}$ as Scale, with the overflow annihilated en route.

Modulus Switching

The modulus switch of Theorem 2.8 scales a ciphertext element by Q'/Q and rounds — a positional operation, and the first casualty of the residue representation. Realized subtractively, however, it needs no positions: for the switch from Q to $Q' = Q/q_{k-1}$,

Algorithm 2.2 SCALE(x, P)

Input: $\mathsf{NTT}([x]_{\mathcal{B}_Q})$ with $|\mathcal{B}_Q| = k$ and a basis $\mathcal{B}_P$ with $|\mathcal{B}_P| = k'$
Output: $\mathsf{NTT}([xP]_{\mathcal{B}_{QP}})$

1: **for** $(x_i, q_i) \in x$ **do**
2: $\quad x_i \leftarrow x_i \cdot P \bmod q_i$
3: **end for**
4: **for** $i \in [0, k')$ **do**
5: $\quad x_{k+i} \leftarrow 0 \bmod p_i$ ▷ Append empty p_i-tower to x
6: **end for**
7: **return** x

a correction $\delta \equiv c \pmod{q_{k-1}}$ makes $c - \delta$ an exact multiple of q_{k-1}, the division becomes a modular multiplication by $[q_{k-1}^{-1}]_{q_i}$ on each remaining limb, and the last limb — identically zero after the subtraction — is dropped [KPZ21]:

$$[c']_{q_i} = \left[q_{k-1}^{-1}\,(c - \delta)\right]_{q_i}, \quad 0 \le i < k - 1 \tag{2.12}$$

The correction is read directly off the limb being dropped: with the centered representative $\delta = [c]_{q_{k-1}}$, subtracting δ moves c to the nearest multiple of q_{k-1}, and (2.12) computes exactly $\lfloor c/q_{k-1} \rceil$. The noise shrinks by the same factor, matching Theorem 2.8.

Approximate Mod Down

The descent of the modulus extension is the same switch performed from QP down to Q, dropping all of $\mathcal{B}_P$ at once. The correction must now be congruent to x modulo the composite P, and it is already present in the operand: the P-limbs hold precisely $[x]_{\mathcal{B}_P}$. What the single-limb switch could read off directly must here be carried across bases — a base conversion [GHS12; HPS18]:

$$\textsc{ApproxModDown}(x) = \left[P^{-1}\,(x - \textsc{FastBConv}([x]_{\mathcal{B}_P}, \mathcal{B}_P, \mathcal{B}_Q))\right]_{\mathcal{B}_Q} \tag{2.13}$$

The conversion being the fast one, the subtracted term is not $[x]_P$ but a lift $[x]_P + \alpha P$ with $|\alpha| \le \lceil k'/2 \rceil$ (2.10), and the extra multiple of P survives the division as an additive error on the result — hence *approximate*:

$$\textsc{ApproxModDown}(x) \equiv \lfloor x/P \rceil - \alpha \pmod{Q}$$

The error exchange is what the temporary extension exists to buy. Everything of value at modulus QP carries a factor P — put there by SCALE or by P-scaled key material — so the operand arriving at the descent has the form $x = Py + e$, with e the noise deposited by the computation at QP. APPROXMODDOWN returns

$$y + \lfloor e/P \rceil - \alpha \pmod{Q}$$

The payload lands at its original scale, the noise arrives divided by P, and the price is an additive term of a few integers. For BGV the subtraction must additionally not disturb the plaintext residue, so the converted correction is chosen congruent to 0 modulo t [KPZ21].

2.3 Gadget Decomposition

Brakerski and Vaikuntanathan [BV11] first proposed decomposing ciphertext elements into small digits as a means of controlling the noise in the scalar multiplication of ciphertexts. The idea has since been generalized into the idea of *gadget decomposition* and been employed in the design of many FHE schemes [BGV11; Bra12; FV12; GSW13; DM14; CKKS16; CGGI18].

Definition 2.20. Digit Decomposition Given an integer $\gamma \in \mathbb{Z}_q$ for a modulus $q \geq 2$, base β and decomposition parameter ℓ, the digit decomposition $\mathsf{Decomp}(\gamma) = (\gamma_0, \gamma_1, \ldots, \gamma_{\ell-1}) \in \mathbb{Z}_\beta^\ell$ outputs an ordered list of ℓ base-β digits satisfying

$$\gamma = \gamma_0 \frac{q}{\beta} + \gamma_1 \frac{q}{\beta^2} + \ldots \gamma_{\ell-1} \frac{q}{\beta^\ell} = \sum_{i=0}^{\ell-1} \gamma_i \frac{q}{\beta^{i+1}}$$

Digit decomposition has two important properties. First, if the digits γ_i of $\mathsf{Decomp}(\gamma)$ are small, then $\langle \mathsf{Decomp}(\gamma), \epsilon \rangle \leq \ell \cdot \max\{\gamma_i \cdot \epsilon_i\} \leq \ell\beta\epsilon \ll \gamma\epsilon$. This is the property that allows some control over the noise growth. Second, two integers can be multiplied through the decomposition. By construction, $\langle \mathsf{Decomp}(\gamma), g \rangle = \gamma$ for the fixed vector $g = (q/\beta, \ldots, q/\beta^\ell)$, meaning $\langle \mathsf{Decomp}(\gamma), g \cdot x \rangle = \gamma \cdot x$ for an arbitrary integer $x \in \mathbb{Z}$. For either of these properties, the particular form of g is unimportant as long as the digits γ_i remain small. Furthermore, as we are generally working with some intrinsic error in FHE, [CGGI18] proposed allowing some error ϵ in the reconstruction, giving the definition from their paper verbatim:

Definition 2.21. (Abstract) Gadget Decomposition An efficient algorithm $\mathsf{Decomp}^{g,\beta,\epsilon}(v)$ is a valid decomposition on the gadget $g \in \mathbb{Z}_q^\ell$ with quality $\beta \in \mathbb{Z}^+$ and precision $\epsilon \in \mathbb{Z}^+$ if and only if for any RLWE sample $v \in R_Q$ it efficiently and publicly outputs a small vector $u \in \mathbb{Z}_q$ such that $\|u\|_\infty \leq \beta$ and $\langle u, g \rangle \leq v + \epsilon$.

– Other (equivalent) form more convenient to work with, where the output of $\mathsf{Decomp}^{g,\beta,\epsilon}(v) \to D(v)$ and $g \cdot u \to P(u)$ are considered.

Definition 2.22. Gadget Property Two vectors $P(x) \in \mathbb{Z}_Q^\ell$ and $D(y) \in \mathbb{Z}_\beta^\ell$ are said to have the *gadget property* if and only if they satisfy (2.14) for any pair of RLWE samples $(x, y) \in R_Q^2$ with precision $\epsilon \in \mathbb{Z}_q^+$ and quality β where $\|D(y)\|_\infty \leq \beta$. If $\epsilon = 0$ then $P(x)$ and $D(y)$ are said to have the *exact* gadget property.

$$\langle P(x), D(y) \rangle = x \cdot y + \epsilon \pmod{Q} \tag{2.14}$$

Lemma 2.23. *The composition of two gadget decompositions is itself a valid gadget decomposition.*

$$\langle P_1(u), D_1(v) \rangle = u \cdot v + \epsilon_1, \quad \langle P_2(u), D_2(v) \rangle = u \cdot v + \epsilon_2 \pmod{Q}$$
$$P_1 \circ P_2 = P_1(P_2(u)) \implies \langle P_1 \circ P_2, D_1 \circ D_2 \rangle = u \cdot v + 2\epsilon \pmod{Q}$$

Proof. Apply the gadget property twice. □

RNS Instantiation

Digit decomposition cannot be applied directly the RLWE sample in an RNS scheme where the system only has access to the residues. When Bajard et. al. implemented the first RNS version of the BV scheme [BEHZ16] they exchanged digit decomposition for CRT decomposition, exploiting the fact that this decomposition is already required by the RNS.

By formulating the CRT as an inner product (A.2), it is immediately clear that CRT with moduli q_i is an exact decomposition with $\beta = \max_i\{q_i\}/2$ (assuming centered residues) on $g = (\hat{q}_0 \cdot [\hat{q}_0^{-1}]_{q_0}, \ldots, \hat{q}_{k-1} \cdot [\hat{q}_{k-1}^{-1}]_{q_{k-1}})$, where $\hat{q}_i = Q/q_i$ and $[\hat{q}_i^{-1}]_{q_i}$ denotes the inverse $\hat{q}_i\hat{q}_i^{-1} \equiv 1 \bmod q_i$. This leads to...

$$P_{\mathsf{HPS}}(a) = (a \cdot [\hat{q}_0^{-1}]_{q_0} \cdot \hat{q}_0, \ldots, a \cdot [\hat{q}_{k-1}^{-1}]_{q_0} \cdot [\hat{q}_{k-1}]_Q) \tag{2.15}$$

$$D_{\mathsf{HPS}}(a) = ([a]_{q_0}, [a]_{q_1}, \ldots, [a]_{q_{k-1}}) \tag{2.16}$$

The HPS gadget is especially powerful, because both (2.15) and (2.16) amount to looking up limbs in RNS (no calculations). $[a]_{q_i}$ is by definition the q_i-tower of a, while

$$[a]_{q_j}[\hat{q}_i^{-1}]_{q_i} \cdot \hat{q}_0 \bmod Q = \begin{cases} 0, & i \neq j \\ [a]_{q_j}, & i = j \end{cases}$$

- Note on HPS gadget = RNS limb lookup → section 3.1
- HPS is exact but noise is large ($\beta = Q/2,\ \epsilon = 0$); can apply digit decomposition to each residue polynomial as per Theorem 2.23, lowering $\beta \to B/2$ and still exact ($\epsilon = 0$) if $B = \lfloor \max q_i/\ell \rfloor + 1$

Definition 2.24. **BV-RNS Gadget** We define the **BV-RNS Gadget** as the double decomposition: Applying digit decomposition to each limb of an RNS polynomial ... termed X

2.3.1 RGSW

The gadget techniques above are used directly in the Gentry-Sahai-Waters (GSW) scheme [GSW13] and its ring variants [DM14; CGGI18], where the ciphertext itself is built around a gadget. GSW came with a homomorphic product between two of its ciphertexts, and TFHE later added an asymmetric product between an RLWE and an (Ring) Gentry-Sahai-Waters (RGSW) ciphertext [CGGI16; CGGI18]. This subsection defines the RGSW ciphertext and both products.

Definition 2.25. **Gadget Matrix** A gadget $g \in \mathbb{Z}_Q^\ell$ is extended to k-dimensional ciphertexts by the block-diagonal gadget matrix $G = I_k \otimes g \in R_Q^{k\ell \times k}$.

Definition 2.26. RGSW Ciphertext Given a gadget $g \in \mathbb{Z}_Q^\ell$ with gadget matrix $G = I_2 \otimes g$, let $Z \in R_Q^{2\ell \times 2}$ be a matrix whose 2ℓ rows are fresh encryptions RLWE(0). An RGSW encryption of $\mu \in R$ is

$$C = Z + \mu \cdot G \in R_Q^{2\ell \times 2} \tag{2.17}$$

An RGSW ciphertext encodes the gadget-scaled messages μg_i row by row. In the top block, μg_i is added to the masked element, replacing the zero message of the underlying RLWE(0); in the bottom block it is added to the public element, so the row decrypts to $-\mu g_i \cdot s$ instead. Every row of C is therefore itself an RLWE ciphertext.

Two RGSW ciphertexts add homomorphically, just as RLWE ciphertexts do: the zero encryptions and the gadget terms add component-wise, so $C_A + C_B$ encrypts $\mu_A + \mu_B$. Linear operations alone, however, are not sufficient for FHE [CGGI18]. The multiplication mirrors the vector–matrix relationship between the two ciphertext types: the RLWE ciphertext $x = (x_0, x_1)$ is mapped by a decomposition function G^{-1} to a row vector of 2ℓ small ring elements, which multiplies the $2\ell \times 2$ matrix C. The only requirement on G^{-1} is that it approximately inverts the gadget matrix from the left,

$$G^{-1}(x) \cdot G = x + e \pmod{Q} \tag{2.18}$$

with $\|G^{-1}(x)\| \leq \beta$ and $\|e\| \leq \epsilon$, where e is the error introduced by the decomposition. No particular decomposition is prescribed: any map satisfying (2.18) yields a valid product, a generality that later chapters exploit by instantiating G^{-1} with different gadgets.

Definition 2.27. External Product Let $x \in R_Q^2$ be an RLWE encryption of $m \in R_t$, let $C \in R_Q^{2\ell \times 2}$ be an RGSW encryption of $\mu \in R$ under the same secret key, and let G^{-1} be a gadget decomposition satisfying (2.18). The external product [CGGI18] is the map

$$\begin{aligned} \boxdot : \mathrm{RLWE} \times \mathrm{RGSW} &\longrightarrow \mathrm{RLWE} \\ (x, C) &\longmapsto x \boxdot C = G^{-1}(x) \cdot C, \end{aligned} \tag{2.19}$$

which returns an encryption of the product of the messages: $\mathrm{RLWE}(m) \boxdot \mathrm{RGSW}(\mu) = \mathrm{RLWE}(m \cdot \mu)$. Correctness follows by inserting (2.17) into (2.19):

$$G^{-1}(x) \cdot C = G^{-1}(x) \cdot Z + \mu \cdot (x + e) \tag{2.20}$$

where $\mu \cdot x$ carries the message and the remaining terms contribute only noise; formal correctness and noise bounds are given in [CGGI18].

Lemma 2.28. Gadget Matrix Construction *Two vectors $P(u), D(v)$ that satisfy the gadget property* (2.14) *define a valid pair of gadget matrices with $g = P(1)$:*

$$G = I_2 \otimes P(1) \tag{2.21}$$

$$G^{-1}(x) = [D(x_0)|D(x_1)] \tag{2.22}$$

satisfying (2.18)*, with $\|G^{-1}(x)\| \leq \beta$ inherited from the quality of D.*

Proof. The block structure reduces the product to one inner product per component, each an instance of (2.14):

$$G^{-1}(x) \cdot G = [D(x_0)|D(x_1)] \begin{pmatrix} P(1) & 0 \\ 0 & P(1) \end{pmatrix}$$

$$\big(\langle D(x_0), P(1)\rangle,\ \langle D(x_1), P(1)\rangle\big) = (x_0 + e_0,\ x_1 + e_1) = x + e \pmod{Q}$$

□

Remark 2.29. The two noise terms of (2.20) behave very differently, as detailed in Appendix B. The term $G^{-1}(x) \cdot Z \leq (a \cdot s + \beta \sum_{i=0}^{2\ell-1} \epsilon_i,\ a)$ amplifies the masking errors of the RGSW operand by the decomposition quality. The precision term e is scaled by μ, which is why μ is typically constrained to $\{0, 1\}$ when an inexact gadget is used. The error of the RLWE operand, by contrast, is never amplified: it is carried into the result scaled only by μ. This asymmetry was observed early on [BV13; AP14; DM14], and makes GSW-style ciphertexts particularly suited to evaluating long chains of products: the noise stays low as long as the right-hand side is fresh, regardless of the error already accumulated on the left.

The original GSW construction defined its homomorphic product directly between two of its ciphertexts [GSW13]. Since every row of an RGSW ciphertext is itself an RLWE ciphertext, this *internal product* is recovered by applying the external product row by row.

Definition 2.30. Internal Product Let $A, C \in R_Q^{2\ell \times 2}$ be RGSW encryptions of $\mu_A, \mu_C \in R$ under the same secret key and gadget, and let A_i denote the i-th row of A. The **internal product** [CGGI18] applies the external product to each row of the left operand:

$$\boxtimes : \text{RGSW} \times \text{RGSW} \longrightarrow \text{RGSW}$$

$$(A, C) \longmapsto A \boxtimes C = \begin{bmatrix} A_0 \boxdot C \\ A_1 \boxdot C \\ \vdots \\ A_{2\ell-1} \boxdot C \end{bmatrix}, \tag{2.23}$$

which returns an encryption of the product of the messages: $\mathrm{RGSW}(\mu_A) \boxtimes \mathrm{RGSW}(\mu_C) = \mathrm{RGSW}(\mu_A \cdot \mu_C)$.

One internal product amounts to 2ℓ external products, and correctness follows row by row: each external product multiplies the content of A_i by μ_C, so the stacked result is again of the form (2.17) with message $\mu_A \cdot \mu_C$.

The asymmetry of Theorem 2.29 applies to each of the 2ℓ rows, and dictates the order in which a chain of products should be evaluated: the noise of the right-hand operand is amplified by the decomposition in every row of the result, while the noise of the left-hand operand is only carried through. When accumulating a running product, the accumulator must therefore be kept on the left with each fresh ciphertext supplied on the right, so that the noise grows additively rather than multiplicatively.

2.4 (Somewhat) Practical Anonymous Router

- Motivation?
- Define practical: instantiation from common FHE primitives (yes) and wrt. time, number of users and throughput (no)
- Section:

Chapter 3
Methodology

A gap needed to be filled: implementing TFHE-style RGSW products (external and internal) in BGV-RNS. As a result of this thesis, a C++ library (`core`) was developed to bring this functionality to the OpenFHE library [BAB+22]. The sPAR protocol has also been implemented in C++ using the RGSW library. I will stress the libraries are separate; the RGSW library can be used on its own as is, and the sPAR library can in theory be used without it — provided a different library is developed to provide the same functionality and API.

This section also explains the set up of the various tests; the numbers from running these tests are presented in chapter 4.

Bitwise Operations

The library assumes the base $B = 2^b$ is a power of two, and ensures this by taking $b = \log B$ as a parameter and derives B. This allows common arithmetic $\mathcal{O}(1)$-performance improvements using bitwise operators:

Table 3.1: Bitwise Operators

Named Function	Operator Notation	Value
LeftShift(x, b)	$x \ll b$	$x \cdot 2^b$
RightShift(x, b)	$x \gg b$	$x/2^b$
And(x, y)	$x \wedge y$	$\sum_i x_i^{(2)} \cdot y_i^{(2)} \cdot 2^i$
Modulo(x, b)	$x \wedge [(1 \ll b) - 1]$	$x \bmod 2^b$

Algorithm 3.1 Center(x, b)

Input: $x, b \in \mathbb{Z}$
Output: $[x]_{2^b} \longmapsto [-2^{b-1}, 2^{b-1})$
1: mask $\leftarrow$ LeftShift$(1, b) - 1$
2: half $\leftarrow$ LeftShift$(1, b - 1)$
3: **return** And$(x + \text{half}, \text{mask}) - \text{half}$

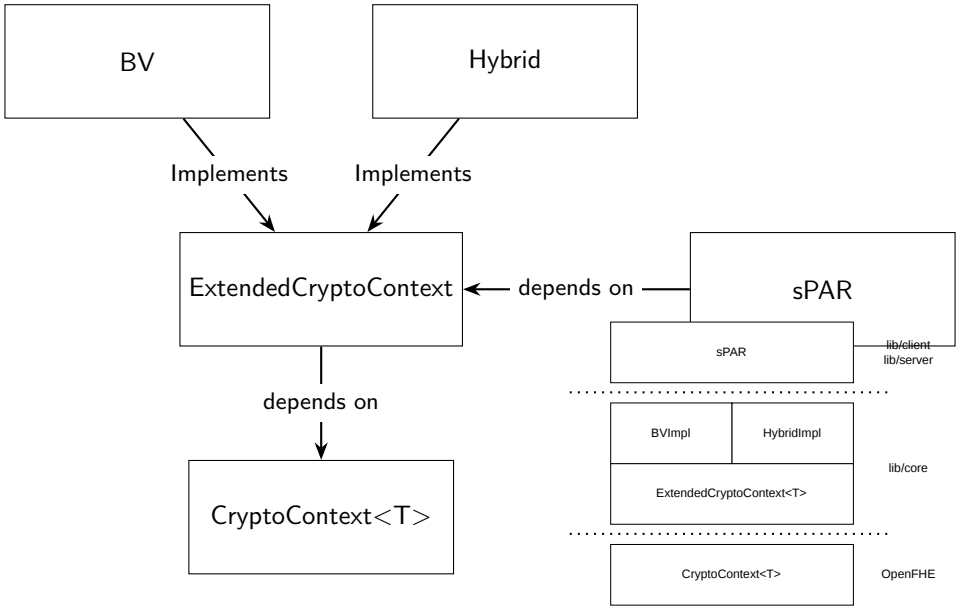
3.1 Implementation Details of the Core Library

The core library, located in the `lib/core` directory of [Ang26] provides an abstract interface named `ExtendendedCryptoContext` which provides a number of RGSW functions, including RGSW encryption, external product and internal product. The advantage of this design is that any number of concrete implementations can be developed for the interface, while any project that requires the provided functionality can import the library and consume the interface without worrying about the implementation of the interface.

The library is built on top of OpenFHE [BAB+22] and the interface reflects that, mirroring the API of OpenFHE. A user who is familiar with OpenFHE should as a result have no problem using this library in their own projects.

Table 3.2: Primary functionality provided by `ExtendedCryptoContext`

EncryptRGSW(pk, pt)	Outputs an RGSW ciphertext
EvalExternalProduct(rlwe, rgsw)	Outputs the external product
EvalInternalProduct(lhs, rhs)	Outputs the internal product



(a) Dependency graph **(b)** Structure (preview)

The `ExtendedCryptoContext` interface comes bundled with two concrete implementations. One is designed around gadget decomposition of the RNS primes as described in section 2.3, and will be referred to as the BV implementation. The other implementation is designed around the hybrid keyswitching mechanism from **??**. Of these, the former should be considered the "canonical" implementation because (1) this design is the most closely related to the external product as discussed in the literature, where this product is traditionally defined using digit decomposition [**examples-of-this**; DP25], and (2) the other implementation does not use true RGSW ciphertexts.

Remark 3.1. The implementations are hard coupled to the BGV-RNS (`CryptoContextBGVRNS`) scheme. As OpenFHE provides a unified interface for both BGV and BFV — and the TFHE products are defined independent of scheme — it would not require much work to add support for BFV to the core library.

- A result of this thesis: a C++ library which extends OpenFHE's BGV implementation to support RGSW operations
- And sPAR which was built on top of this library
- The code base is split into three sections found in the `lib` directory:

  1. `core`extends OpenFHE's `CryptoContext<T>` with the operations:
     - `EncryptRGSW` outputs the RGSW encryption of the input plaintext
     - `EvalExternalProduct` takes an RLWE and RGSW ciphertext

    - `EvalInternalProduct` takes two RGSW ciphertexts
    - Supporting functions like AddRGSW and MultRGSWPlaintext etc.

2. `server`builds the sPAR server functions on top of `core`

3. `client`builds the sPAR client functions on top of `core`

- There are also two auxiliary directories `benchmarks` and `tests` that contain code to benchmark and unit test functions, respectively.
- And `scripts` contain python code for running the estimator

3.1.1 Concrete Implementation

- Digit decomposition of each residual limb
- Two functions that output $P(v)$ and $D(u)$ with quality β s.t. $\|D(u)\|_\infty \leq B/2 = \lceil \max q_i/(2\ell) \rceil$
- The outputs has $k \times \ell$ entries, where k is the number of RNS moduli (corresponding to the first decomposition) and ℓ is the number of base B digits
- Takes only ℓ as an input parameter and computes $\log B = \lfloor \max\{\log(q_i)\}/\ell \rfloor + 1$
- The base $B = \lfloor \max q_i \rfloor$ is chosen automatically so to be the smallest B that covers the largest RNS prime with ℓ digits when the program starts. This makes the gadget exact, even if the q_i are generated at different orders (in OpenFHE the first prime is typically larger than the other primes for some reason).

RGSW Encryption

- `ExtendedContextBV` implements the double decomposition; digit decomposing each RNS limb
- Let $P_g(v) = g \otimes v$ and $P_{\mathsf{BV}}(v) = P_g \circ P_{\mathsf{HPS}}(v) = g \otimes (P_{\mathsf{HPS}}(v))$

$$C = Z + \mu G = Z + \mu \cdot (I_2 \otimes P_g(P_{\mathsf{HPS}}(1)) = Z + (I_2 \otimes P_g(P_{\mathsf{HPS}}(\mu))$$
$$P_g(P_{\mathsf{HPS}}(\mu)) = g \otimes ([\mu]_{q_0}, \ldots, [\mu]_{q_{k-1}})$$
$$([\mu g]_{q_0}, \ldots [\mu g]_{q_{k-1}}) \text{ where } \mu g = (\mu, \mu B, \ldots, \mu B^\ell)$$
$$([\mu]_{q_0}, [\mu B]_{q_0}, \ldots, [\mu B^\ell]_{q_{k-1}}) \in \mathbb{Z}_Q^{k \cdot \ell}$$

- Observe that this is the same information as $(\mu, \mu B, \ldots, \mu B^\ell)$
- Because the i-th tower of μ is $[\mu]_{q_i}$, the HPS gadget row j gives $[[\mu]_{q_i}]_{q_j} = 0 \forall i \neq j$

$$\left[\begin{array}{ccc|ccc|ccc} [\mu]_{q_0} & \cdots & [\mu B^{\ell-1}]_{q_0} & 0 & \dots & 0 & 0 & \dots & 0 \\ 0 & \dots & 0 & [\mu]_{q_1} & \cdots & [\mu B^{\ell-1}]_{q_1} & 0 & \dots & 0 \\ \vdots & \ddots & \vdots & \vdots & \ddots & \vdots & \vdots & \ddots & \vdots \\ 0 & \dots & 0 & 0 & \dots & 0 & [\mu]_{q_{k-1}} & \cdots & [\mu B^{\ell-1}]_{q_{k-1}} \end{array}\right]$$

- Ignore all the 0 elements, then we get the exact definition of the RNS form of μ, μB, and so on
- However, each row of the RGSW must still encode only 1 tower from each of the ℓ polynomials

$$P_{\mathsf{BV}}(\mu) \overset{\text{flatten}}{\longmapsto} \begin{bmatrix} [\mu]_{q_0} & \cdots & [\mu B^{\ell-1}]_{q_0} \\ [\mu]_{q_1} & \cdots & [\mu B^{\ell-1}]_{q_1} \\ \vdots & \ddots & \vdots \\ [\mu]_{q_{k-1}} & \cdots & [\mu B^{\ell-1}]_{q_{k-1}} \end{bmatrix} = ([\mu]_{\mathcal{B}_Q}, \dots, [\mu B^{\ell-1}]_{\mathcal{B}_Q})$$

- Multiplying μ by a factor of B^i for a single i can be done in NTT
- Applying this to $Z + P_{\mathsf{BV}}(\mu)$ allows parallelization on $k\ell$ threads
- $\mathcal{B}_Q$, ℓ and B are all known/global variables tied to the context

Algorithm 3.2 Parallel RGSW-BV Encryption

Input: Decomposition parameter ℓ, base B
Input: RNS polynomial $[\mu]^{\mathsf{NTT}}_{\mathcal{B}_Q} \in \mathbb{Z}_{q_0} \times \cdots \times \mathbb{Z}_{q_{k-1}}$
Output: $\mathsf{RGSW}(\mu)$

1: Initialize empty array C with $2k\ell$ slots
2: **for** $r \leftarrow [0, k\ell)$ **in parallel do**
3: $\quad (i, j) \leftarrow (r \bmod \ell, \lfloor r/\ell \rfloor)$
4: $\quad mg \leftarrow \textsc{TowerMul}(\mu, j, B^i)$
5: $\quad C_r \leftarrow \textsc{Encrypt}(0) + (mg, 0)$
6: $\quad C_{r+\ell} \leftarrow \textsc{Encrypt}(0) + (0, mg)$
7: **end for**
8: **return** C

- Pre-computed value: $\textsc{GetPower}(i, j) = B^i \bmod q_j$ where the modulo is just to keep numbers smaller in memory, negligible for performance
- Then $\textsc{TowerMul}(\mu, j, B^i)$ is really just extracting the $j-th$ limb, multiplying by the pre-computed value $B^i \bmod q_j$ calculate the product modulo q_j before the tower is inserted back into μ

External Product

- **IMPORTANT:** decomposition determines the quality β, so the order of operations does matter for noise growth!
- Decomposition used = per limb digit extraction
- Must choose to decompose into $[-B/2, B/2)$ or $[0, B)$
  - Centered has lower noise as the quality is $\beta = B/2$ but more complicated
  - Unsigned is far simpler and easily parallelizable but $\beta = B$
- Centered representation creates an order; parallelization requires *pre-processing*

$$\text{Let } offset = \frac{B}{2} \cdot \frac{B^\ell - 1}{B - 1} \text{ be a global (pre-calculated) value}$$

Algorithm 3.3 Decompose wrt. B (TODO: Signed digit decomposition)

Input: RNS polynomial $[x]_{\mathcal{B}_Q} \in \mathbb{Z}_{q_0} \times \mathbb{Z}_{q_1} \times \cdots \times \mathbb{Z}_{q_{k-1}}$, base $B = 2^b$, and decomposition parameter ℓ
Output: ℓ RNS "digit" polynomials $\mathbf{d} = (d_0, d_1, \ldots, d_{k\ell-1}) \in \mathbb{Z}_B^{k\ell}$

1: $a' \leftarrow a$
2: **for** $j = 0$ to $\ell - 1$ **do**
3: $\quad d_j \leftarrow a' \pmod B$ ▷ Extract the lower bits
4: $\quad$ **if** $d_j \geq B/2$ **then**
5: $\quad\quad d_j \leftarrow d_j - B$ ▷ Shift to signed range
6: $\quad$ **end if**
7: $\quad a' \leftarrow \frac{a' - d_j}{B}$ ▷ Shift down and apply the carry
8: **end for**
9: **return d**

- Uses the standard external product function **??**
- Or a slightly parallelized version: the decompose function runs in parallel, not much gain from parallizing further, the additions are already cheap/fast-ish.

Internal Product

- Trivially implemented as repeated external products as per the definition Theorem 2.30

SIMD Batching

- Assuming parameters that support SIMD batching are set

- Requires using `MakePackedPlaintext` instead of `MakeCoefPackedPlaintext` in OpenFHE
- This allows up to N RGSW/External/Internal prodcuts simultaneously at no additional cost compared to the same parameters with MakeCoefPacked-Plaintext(.)

3.2 Implementation Details of sPAR

My instantiation of sPAR contains some differences from the specification in [DP25]:

- Some functions are missing: bucket sorting, all extras such as equivocation protection etc.
- Sorting is negligible, estimated < 100 ms according to [**<empty citation>**] and highly parallelizable.
- Write step has been separated into two steps
- Uses a procedural programming paradigm, meaning the library implements the required functions but does not provide a full state-maintaining implementation of the server nor client.
- The benchmarks and testing framework is what actually composes the protocol, using the functions from this library.
- This was done so that the functionality is not tightly coupled to the benchmarking without spending time implementing actual communication which would require additional work: serializing data, error handling, and coordinating multiple machines.

3.2.1 Server Write Step

- Most important algorithm; does the actual homomorphic sorting
- Some changes compared to [DP25]
  - Splits the write step and pre-processing step
  - HasWrite is inverted i.e. "notHasWrite"
  - Uses external products to handle deep circuits as [DP25] recommends.
  - Not implementable using only external products: must use internal products
  - As the sever does not care about the privacy of its users, and we can assume it is actively attempting to break privacy, to manage noise **the server does not actually encrypt the initial entries of L, I and $hasNotWritten$.** They are in the correct form but generated using $\epsilon = 0, a = 0$. This has a very minor impact on noise but worth noting, major impact on setup phase as it does not spend any time on encrypting or generating random values.
  - Variables:

    * $z_{i,d}$, $I_{i,k}$, $\mathsf{hasNotWritten}$ are $\mathrm{RGSW}(m), m \in \{0,1\}^N$
    * $L_{i,k} = \mathrm{RLWE}(m), m \in R_t$
  - Swapping order so $h = (\mathsf{hasNotWritten} \boxtimes I_{i,k}) \boxtimes z_{i,d})$ saw a 30 to 40 bit improvement in the noise of SERVER::WRITE

Algorithm 3.4 `Server::Write`

Input: Reference to state matrices I and L
Input: Encrypted RGSW value V_r
Input: RGSW indices A_0, A_1 and A_2 where A_j is $\log \eta$ encrypted bits $\{c_{j,0}, \ldots, c_{j,\eta-1}\}$ for $j \in [0,3)$
Output: Updates the state matrices I and L
Output: Encrypted boolean indicating wether the operation was successful

1: $\mathsf{hasNotWritten} \leftarrow$ ENCRYPTRGSW(1)
2: **for** $d \leftarrow \{0,1,2\}$ **do**
3: **for** $k \leftarrow \{0,1,2\}$ **do**
4: **for** $i \leftarrow [0,\eta)$ **do**
5: $h \leftarrow (\mathsf{hasNotWritten} \boxtimes I_{i,k}) \boxtimes z_{i,d})$
6: $L_{i,k} \leftarrow L_{i,k} + V_r \cdot h$
7: $I_{i,k} \leftarrow I_{i,k} - h$
8: $\mathsf{hasNotWritten} \leftarrow \mathsf{hasNotWritten} + h$
9: **end for**
10: **end for**
11: **end for**
12: **return** $\mathsf{hasNotWritten}$

Remark 3.2. From a programming perspective, the choice of $K = 3$ and $D = 3$ is arbitrary. The library allows a developer to configure different values for K and D as compile-time constants, with no impact on the size of the compiled binary/program. They are implemented as C++ generics, also known as template specialization.

3.3 Experimental Setup

sPAR

The design To simulate communication between parties, a single "all-knowing" orchestrator with access to the secret key.

- An "all-knowing" orchestrator performs the setup and 1 full round
- In `benchmark` dir
  - Takes the number of users η as an input
  - Every "user" generates a private key share, and corresponding public key share
  - The public key shares are all synthesized to make one large public key
  - For keeping track, each user has a unique ID
  - Each user encodes their ID in an RGSW using the public key, and sends that message to the server
  - Each user also sends 3 random indices
  - After receiving all messages, the server performs the write operation on each message
- Setup phase (considered "free")
  - Generate public key shares and attributes them to η virtual users
  - Hybrid setup phase is more complicated; requires additional MPC to generate a joint public key in QP used to encrypt RGSW ciphertexts

Unit Tests + Benchmarks

- To empirically evaluate correctness and catch bugs early, the functionality of the program is unit tested
- The RGSW encryption + TFHE operations are tested for plaintext 0 and 1, and a larger message
- SIMD and regular variants are tested
- For sPAR protocol, the number of users is parameterized over η
- `test` checks for correctness at every stage, no failures means the protocol can handle η users
- Even though q_i are randomly generated each time, its empirically clear there is no difference between runs, no matter what q_i are (as long as $\|Q\|$ is at the same bit size)
- Difference between benchmarks and unit tests:
  - Benchmarks are optimized for speed, and do not perform any correctness checks (which could affect performance). Functions only need to run once and total time can be extrapolated: for the client encryption/decryption which can be run in parallel over/independently by each user, and the

server/final decryption, the one-shot runtime *is* the correct runtime. For the server write, we only care about the per-user runtime anyway.
  - Unit tests are not optimized for speed (in the compilation etc.) and does perform correctness checks multiple times, at different levels; that way, the library provides reasonable diagnostics. Not just that an operation failed, but also *why*/where it failed.
  - Unit tests and benchmarks match in parameter and other code, so a successful unit test empirically validates correctness while the benchmarks provides real performance numbers in a production setting.

Noise Analysis

- For testing RGSW products, error is checked directly
- Simulation/benchmark/testing environment always has access to the secret key

$$x = (c_0, c_1) = (c_1 s + \Delta m + \epsilon, c_1) \tag{3.1}$$

$$\implies \epsilon = c_0 - c_1 s - \Delta m, \quad m = \textsc{Decrypt}(x, s) \tag{3.2}$$

- RGSW is an 'intermediate' object used in calculations, in some cases (when the result is an RGSW e.g., the internal product, notHasWritten in the server) two methods exist to evaluate noise:
  1. The first (or ℓ-th) row encodes $\mu \cdot 1$, so we can extract only this row and check it's error
  2. We calculate the external product with RLWE(1), and this ciphertext can be noiseless as to not add any noise.
- I use the latter method because the first method relies on the fact that a gadget $g = (1, \dots)$ containing 1 is used, which is not necessarily always the case, even though this is traditionally the case.

Chapter 4
Results and Discussion

This section begins by presenting the parameters used in benchmarking, then ...

I will close the section by discussing the results and suggest a new approach...

4.1 Parameters

I found that the parameters suggested in [DP25] results in too much noise, and are thus not capable of running the full sPAR protocol without decryption failure. Rather than tweaking these parameters until the protocol could run, I instead aim to fully utilize the RNS context by using parameters that enable SIMD batching.

Parameters were thus selected to provide $\lambda \geq 128$ bits of security at the largest possible Q while supporting SIMD batching (t prime s.t. $t \equiv 1 \bmod N$), then ℓ was tweaked until the sPAR protocol runs without decryption failure. The RNS primes are set slightly below 64 bits to minimize k, with the first prime q_0 being slightly larger than the following primes by convention?? I found that this left much headroom, prompting me to lower k until $\ell = 2$ could no longer sustain the noise. The resulting parameters, listed in Table 4.1, yield a security parameter of 339.6 bits [APS15] for the underlying RLWE scheme.

Table 4.1: Base parameters for BV-RNS

Parameter	**Value**
decomposition (ℓ, B)	$(3, 2^{21})$
number of RNS moduli k	4
first mod size $\log q_0$	60
scaling mod size $\log q_i,\ i > 0$	59
ciphertext modulus order $\log Q$	2^{296}
standard deviation σ	3.19
plaintext modulus t	65537
polynomial degree N	16385
security parameter λ	339.6

- Explain difference in size between q_0 and q_i
- Explain q_i are random 59-bit primes so only their order is listed
- Data per message $= N \log(t)/8 \approx 16385 \cdot (16)/8 = 32770$ bytes ≈ 32 Kb
- sPAR data per msg $= 2048 * 8/8 = 2048 = 2$ Kb
- For performance, the key is generally to minimize $k\ell$, in this case $2k\ell = 2 \cdot 4 \cdot 3 = 2 \cdot 12 = 24$ rows per RGSW!
- Note for self: if larger params is 16 times slower than expected, throughput is the same

4.2 Noise Analysis

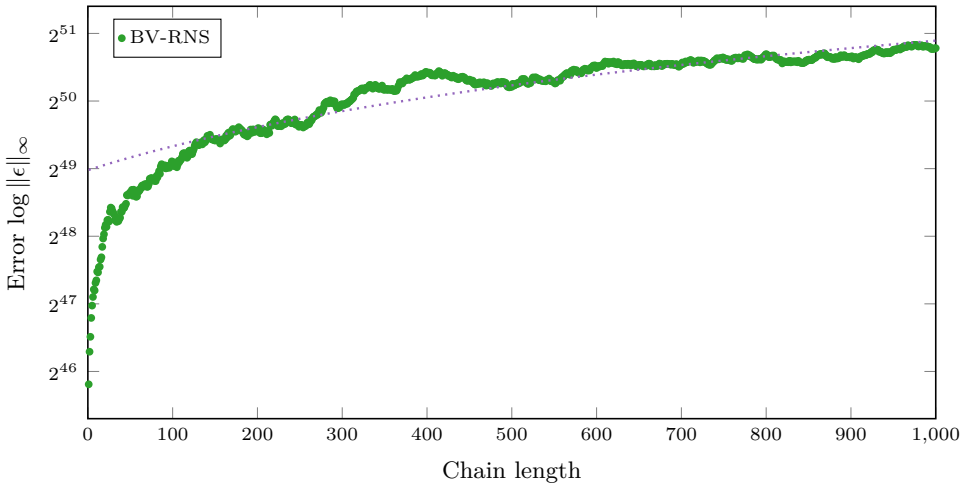
- I present the noise growth of various functions
- Using the max-norm $\|\epsilon\|_\infty = \max_i\{\epsilon_i\}$ etc.
- The absolute value does not really matter; we care more about how many bits of error
- Noise plots are therefore semi-logarithmic, with the y-axis displaying $\log(v)$

4.2.1 Core Library

External Product

- Note: LHS is the accumulated value, RHS is always fresh (due to known decomposition asymmetry that should be discussed in Theorem 2.29)
- Error is additive, less than 51 bits of noise for 1000 chained operations
- To illustrate linear growth: linear regression is plotted on the graph as a dotted line
- Use linear regression to determine max chain length: ...
- Maybe: show max chain length if RHS is accumulated noise

Figure 4.1: RNS-BV noise growth in chained external products

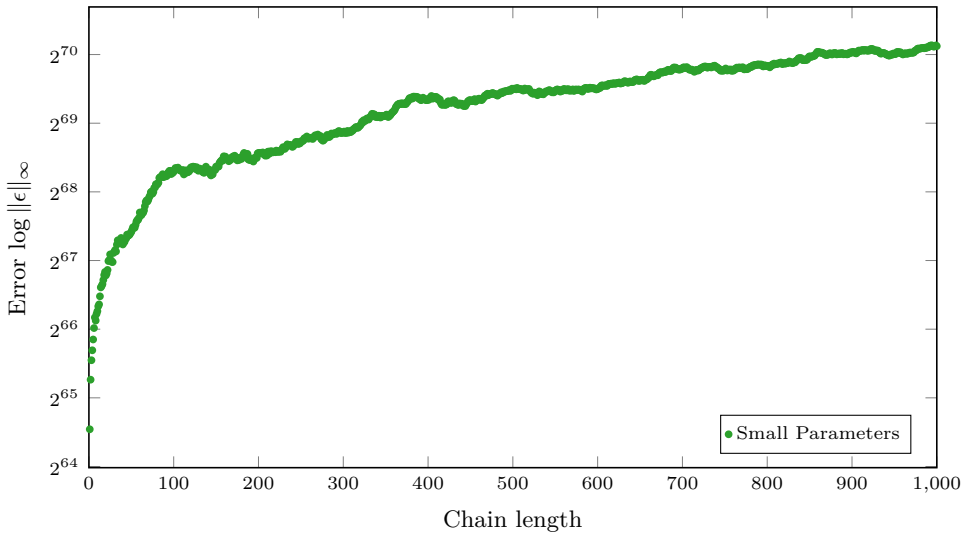


Internal product

- Note: LHS is still the accumulated value, RHS is always fresh
- Error is still additive, but each internal product is several external products
- Less than 71 bits of noise for 1000 chained operations

– This is much larger, on the order of 2^{20}

Figure 4.2: RNS-BV noise growth in chained internal products TODO: Overlay graph of using accumulated result on rhs

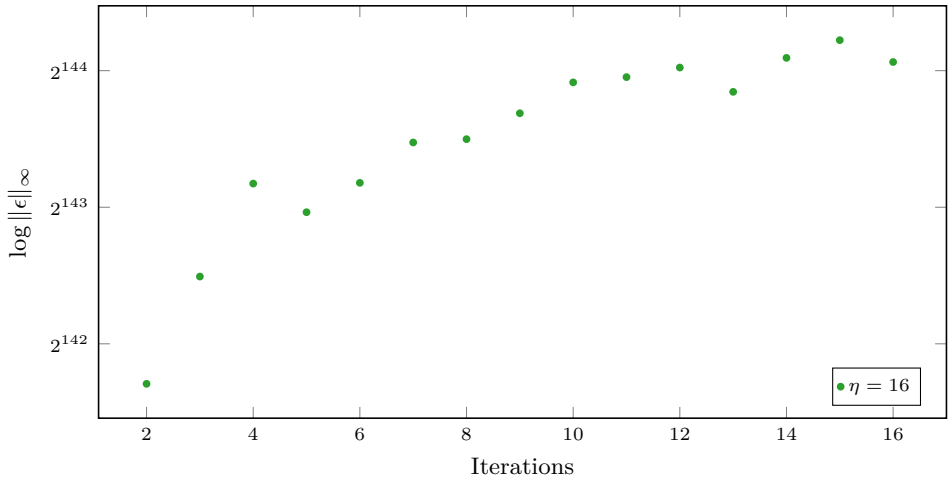


– $1.08139e18 \cdot x + 2.1953e20 \leq 2^{180} \implies x \leq 1.41715 \cdot 10^{36}$

4.2.2 sPAR

– If the parameters support even the base case of $\eta = 2$ users, then the scheme can support a nearly arbitrary amount of users with the same parameters
– Noise is a big problem due to the internal product
– Getting enough noise headroom at $\lambda = 128$ requires very large parameters, making the protocol much slower than anticipated

Figure 4.3: `Server::Write`



- Linear regression: $1.47533e42x + 4.7166e42$
- Noise ceiling: 180 bits
- Assuming same growth: $1.47533e42\eta + 4.7166e42 \leq 2^{180} \implies \eta \leq 1.03875 \cdot 10^{12}$
- Noise growth is probably larger as η increases; have to verify

Multi-Party FHE Noise

- This section is separated from the
- My instantiation uses the default OpenFHE threshold FHE, which represents a practical (easy to set up) but not necessarily the state-of-the-art of MPC.

4.3 Performance Benchmarks

- Run on AMD Ryzen 5 5600X CPU 6-core @ 3.7 GHz with 32Gb RAM in WSL2 Ubuntu 24.04 LTS Linux

4.3.1 Core Library

Table 4.2: RGSW Operations (todo: update numbers)

Operation	Time
Encrypt RGSW	5.08 ms
External Product	1.66 ms
Internal Product	20.24 ms

- The internal product scales exponentially as it performs $2k\ell$ external products, which in turn performs $2k\ell$ multiplications = $\mathcal{O}(4k^2\ell^2)$
- Since GHS specifically only has only 2 rows, the scaling is a set factor of approx. 2 instead of $2k\ell$

4.3.2 sPAR

Table 4.3: One round of the full sPAR protocol, not adjusted to per user except Write, 6 threads

η	Total	Encrypt	Write	Partial Dec.	Server Dec.
2	8620 ms	402.53 ms	8.06798 s	140.60 ms	5.02677 ms
4	17984 ms	1652.05 ms	15.9965 s	425.13 ms	10.783 ms
8	41232 ms	6745.00 ms	33.0402 s	1472.59 ms	28.1619 ms
16	98332 ms	26382.50 ms	67.368 s	5005.41 ms	60.5488 ms

- **Discussions:** If write can use SIMD batching the write loop can be divided by up to $N \implies$ 4.675 ms, 18.798 ms, 74.69 ms per round, respectively, but the other functions must also be scaled up..
- TODO: run single-threaded, adjust to η i.e. divide by η for encrypt, write, partial decrypt

4.3.3 Multi-Threaded Performance

- Works now, but only some functions (EncryptRGSW, BV Decompose) are optimized to take advantage of multi-threading.
- $\leq k\ell$ threads saturate, so $k = 2, \ell = 3$ has optimal performance at 6 threads.
- Ideally more threads would used to perform multiple coefficients simultaneously, but this is not implemented.

Table of improvements +X% at 1 to 6 threads

4.4 Discussions

- The results show that number of users is not a problem; noise is
- Once noise is controlled, many users can be handled due to the additive scaling of the RGSW products
- GHS improvement over BV even adjusted to same security parameter, but would require a solution to the internal product

4.4.1 Theoretical Hybrid Improvement

An alternative to gadget decomposition used in RNS key-switching is *hybrid* keyswitching [GHS12; KPZ21].

- This approach nearly fits inside the gadget decomposition framework established in section 2.3, as $\langle u \cdot P^{-1}, v \cdot P \rangle = u \cdot v \cdot P^{-1} \cdot P = u \cdot v \bmod QP$
- This actually defines a valid external product, the problem is that the P-scaling is consumed after this operation

Definition 4.1. Hybrid-RGSW Ciphertext An RGSW ciphertext that encodes μP in the extended

$$C_{QP} = Z_{QP} + \mu \begin{pmatrix} P & 0 \\ 0 & P \end{pmatrix} \pmod{QP} \tag{4.1}$$

$$\textsc{Hybrid}\text{RGSW}(b) = \text{RGSW}(b \cdot P) \tag{4.2}$$

Algorithm 4.1 Hybrid-RGSW Encryption

Input: Extended basis $\mathcal{B}_P$
Input: RNS polynomial $[\mu]^{\mathsf{NTT}}_{\mathcal{B}_Q} \in \mathbb{Z}_{q_0} \times \cdots \times \mathbb{Z}_{q_{k-1}}$
Output: RGSW$(\mu \cdot P)$

```
Initialize empty array C with 2kℓ slots
for r ← [0, kℓ) in parallel do
    z ← HE.Encrypt(0, pk_{B_QP})
    mG ← P · BaseExtend(m, Q, P)
    C_r ← HE.Enc(0) + (mg, 0)
    C_{r+ℓ} ← HE.Enc(0) + (0, mg)
end for
return C
```

Definition 4.2. Hybrid External Product Is valid...

Lemma 4.3. *The external product RGSW $\boxdot$ HybridRGSW is a valid product...*

Proof.

$$x_Q \boxdot C_{QP} = P^{-1}(x_q \cdot Z_{QP} + \mu \begin{pmatrix} P & 0 \\ 0 & P \end{pmatrix}) \pmod{QP} \tag{4.3}$$

□

- This method shows far better noise handling and
- Show graphs for same parameters but $|P| \approx |Q|$ i.e. $d_{num} = 1$ and security parameter smaller but still ≥ 128 bits

Algorithm 4.2 RLWE $\boxdot$ Hybrid-RGSW

Input: Extended basis $\mathcal{B}_P$
Input: RNS polynomial $[\mu]^{\mathsf{NTT}}_{\mathcal{B}_Q} \in \mathbb{Z}_{q_0} \times \cdots \times \mathbb{Z}_{q_{k-1}}$
Output: RGSW$(\mu \cdot P)$
1: Initialize empty array C with $2k\ell$ slots
2: **for** $r \leftarrow [0, k\ell)$ **in parallel do**
3: hm
4: **end for**
5: **return** C

– Problem: internal product does not work with both sides as Hybrid-RGSW and one P is consumed each external product!

Definition 4.4. Hybrid Internal Product I define the *hybrid internal product* as

$$\boxtimes_H \;:=\; \text{RGSW}(a) \boxtimes_H \text{RGSW}(b \cdot P) = \text{RGSW}(a \cdot b) \tag{4.4}$$

Conjecture 4.5. Mixed Internal Product *Test test*

Algorithm 4.3 Modified

Input: Extended basis $\mathcal{B}_P$
Input: RNS polynomial $[\mu]^{\mathsf{NTT}}_{\mathcal{B}_Q} \in \mathbb{Z}_{q_0} \times \cdots \times \mathbb{Z}_{q_{k-1}}$
Output: RGSW$(\mu \cdot P)$
1: Initialize empty array C with $2k\ell$ slots
2: **for** $r \leftarrow [0, k\ell)$ **in parallel do**
3: $h \leftarrow (I_{i,k} \boxtimes \mathsf{nhw})\tilde{} \boxtimes_H z_{i,d}$
4: **end for**
5: **return** C

– Assuming Algorithm 3.4 is implemented using only external products, or the internal product can be implemented in the hybrid scheme, we see that the hybrid approach is generally faster and has better noise properties

Chapter 5

Conclusion and Future Work

Introduction...

5.1 Conclusion

- Achieved better time than expected with GHS
- Large noise
- Number of users

5.2 Future Work

- BGV SIMD capabilities [SV11] not utilized in write loop; `Server::Write` can perform N rounds simultaneously, but does this translate into higher throughput? A constant polynomial is the same object in SIMD batching and as a regular polynomial, so if write is changed to output binary $\{0, 1\}$ it should be possible with:
  - For each $i \in [0, N)$
  - mask $\leftarrow e_i = (0, 0, \ldots, 1, \ldots, 0)$ where only the i-th element is 1
  - isolated $\leftarrow h \cdot$ mask
  - msg
- AVX512 not tested because time + no access to a computer that supports this instruction set; this would decrease the RNS primes to 48 bits, increasing performance and security, decreasing noise headroom
- RGSW/packed RLWE bandwidth optimization; originally planned, dropped due to time constraints, requires more complicated setup phase as $\mathsf{RGSW}s$ is required [CCR19; LY25]
- BFV implementation should be easy as the library sits on top of OpenFHE, it should be as easy as swapping out a few lines of code. Since BFV places its messages in the most significant bits, the decomposition gadget could be made inexact as with the powers of base, the least significant digits are dropped when ℓ is too small for B^ℓ to cover q
- The full sPAR protocols contains additional things such as equivocation protection, sorting buckets etc.

References

[Ang26] I. Angelo, *Implementation of (Somewhat) Practical Anonymous Router*, version 1.0.0, Jul. 2026. [Online]. Available: https://github.com/ImreAngelo/sPAR.

[AP14] J. Alperin-Sheriff and C. Peikert, *Faster bootstrapping with polynomial error*, Cryptology ePrint Archive, Paper 2014/094, 2014. [Online]. Available: https://eprint.iacr.org/2014/094.

[APS15] M. R. Albrecht, R. Player, and S. Scott, *On the concrete hardness of learning with errors*, Cryptology ePrint Archive, Paper 2015/046, 2015. [Online]. Available: https://eprint.iacr.org/2015/046.

[BAB+22] A. A. Badawi, A. Alexandru, *et al.*, *OpenFHE: Open-source fully homomorphic encryption library*, Cryptology ePrint Archive, Paper 2022/915, 2022. [Online]. Available: https://eprint.iacr.org/2022/915.

[BEHZ16] J.-C. Bajard, J. Eynard, *et al.*, *A full RNS variant of FV like somewhat homomorphic encryption schemes*, Cryptology ePrint Archive, Paper 2016/510, 2016. [Online]. Available: https://eprint.iacr.org/2016/510.

[BGV11] Z. Brakerski, C. Gentry, and V. Vaikuntanathan, *Fully homomorphic encryption without bootstrapping*, Cryptology ePrint Archive, Paper 2011/277, 2011. [Online]. Available: https://eprint.iacr.org/2011/277.

[Bra12] Z. Brakerski, *Fully homomorphic encryption without modulus switching from classical GapSVP*, Cryptology ePrint Archive, Paper 2012/078, 2012. [Online]. Available: https://eprint.iacr.org/2012/078.

[BV11] Z. Brakerski and V. Vaikuntanathan, *Efficient fully homomorphic encryption from (standard) LWE*, Cryptology ePrint Archive, Paper 2011/344, 2011. [Online]. Available: https://eprint.iacr.org/2011/344.

[BV13] Z. Brakerski and V. Vaikuntanathan, *Lattice-based FHE as secure as PKE*, Cryptology ePrint Archive, Paper 2013/541, 2013. [Online]. Available: https://eprint.iacr.org/2013/541.

[CCR19] H. Chen, I. Chillotti, and L. Ren, "Onion ring oram: Efficient constant bandwidth oblivious ram from (leveled) tfhe", in *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security*, ser. CCS '19, London, United Kingdom: Association for Computing Machinery, 2019, pp. 345–360. [Online]. Available: https://doi.org/10.1145/3319535.3354226.

[CGGI16] I. Chillotti, N. Gama, *et al.*, *Faster fully homomorphic encryption: Bootstrapping in less than 0.1 seconds*, Cryptology ePrint Archive, Paper 2016/870, 2016. [Online]. Available: https://eprint.iacr.org/2016/870.

[CGGI18] I. Chillotti, N. Gama, *et al.*, *TFHE: Fast fully homomorphic encryption over the torus*, Cryptology ePrint Archive, Paper 2018/421, 2018. [Online]. Available: https://eprint.iacr.org/2018/421.

[CKKS16] J. H. Cheon, A. Kim, *et al.*, *Homomorphic encryption for arithmetic of approximate numbers*, Cryptology ePrint Archive, Paper 2016/421, 2016. [Online]. Available: https://eprint.iacr.org/2016/421.

[DM14] L. Ducas and D. Micciancio, *FHEW: Bootstrapping homomorphic encryption in less than a second*, Cryptology ePrint Archive, Paper 2014/816, 2014. [Online]. Available: https://eprint.iacr.org/2014/816.

[DP25] D. Das and J. Park, *sPAR: (somewhat) practical anonymous router*, Cryptology ePrint Archive, Paper 2025/860, 2025. [Online]. Available: https://eprint.iacr.org/2025/860.

[FV12] J. Fan and F. Vercauteren, *Somewhat practical fully homomorphic encryption*, Cryptology ePrint Archive, Paper 2012/144, 2012. [Online]. Available: https://eprint.iacr.org/2012/144.

[Gen09] C. Gentry, "A fully homomorphic encryption scheme", Ph.D. dissertation, Stanford University, 2009. [Online]. Available: https://crypto.stanford.edu/craig.

[GHS12] C. Gentry, S. Halevi, and N. P. Smart, *Homomorphic evaluation of the AES circuit*, Cryptology ePrint Archive, Paper 2012/099, 2012. [Online]. Available: https://eprint.iacr.org/2012/099.

[GSW13] C. Gentry, A. Sahai, and B. Waters, *Homomorphic encryption from learning with errors: Conceptually-simpler, asymptotically-faster, attribute-based*, Cryptology ePrint Archive, Paper 2013/340, 2013. [Online]. Available: https://eprint.iacr.org/2013/340.

[HPS18] S. Halevi, Y. Polyakov, and V. Shoup, *An improved RNS variant of the BFV homomorphic encryption scheme*, Cryptology ePrint Archive, Paper 2018/117, 2018. [Online]. Available: https://eprint.iacr.org/2018/117.

[KPZ21] A. Kim, Y. Polyakov, and V. Zucca, *Revisiting homomorphic encryption schemes for finite fields*, Cryptology ePrint Archive, Paper 2021/204, 2021. [Online]. Available: https://eprint.iacr.org/2021/204.

[LY25] K. H. Lee and J. W. Yoon, *Homomorphic field trace revisited : Breaking the cubic noise barrier*, Cryptology ePrint Archive, Paper 2025/1088, 2025. [Online]. Available: https://eprint.iacr.org/2025/1088.

[RAD78] R. L. Rivest, L. Adleman, and M. L. Dertouzos, "On data banks and privacy homomorphisms", in *Foundations of Secure Computation*, 1978, pp. 169–180.

[SEV+15] Y. Sun, A. Edmundson, *et al.*, "RAPTOR: Routing attacks on privacy in tor", in *24th USENIX Security Symposium (USENIX Security 15)*, Washington, D.C.: USENIX Association, Aug. 2015, pp. 271–286. [Online]. Available: https://www.usenix.org/conference/usenixsecurity15/technical-sessions/presentation/sun.

[SV11] N. P. Smart and F. Vercauteren, *Fully homomorphic SIMD operations*, Cryptology ePrint Archive, Paper 2011/133, 2011. [Online]. Available: https://eprint.iacr.org/2011/133.

[SW21] E. Shi and K. Wu, *Non-interactive anonymous router*, Cryptology ePrint Archive, Paper 2021/435, 2021. [Online]. Available: https://eprint.iacr.org/2021/435.

Appendix A
Number Theory

A.1 Chinese Remaineder Theorem

Definition A.1. Chinese Remainder Theorem co-prime moduli etc.

$$\begin{aligned} x &\equiv x_0 \pmod{q_0} \\ &\vdots \\ x &\equiv x_{k-1} \pmod{q_{k-1}} \end{aligned}$$

$$\text{Let } Q = \prod_{i=0}^{k-1} q_i,\ \hat{q}_i = \frac{Q}{q_i} \text{ and } \hat{q}_i^{-1} \text{ be the inverse s.t. } \hat{q}_i^{-1}\hat{q}_i \equiv 1 \pmod{q}_i$$

$$\text{Then CRT states } x = \sum_{i=0}^{k-1} x_i \cdot \hat{q}_i \cdot \hat{q}_i^{-1} \pmod{Q} \tag{A.1}$$

$$x = \langle \{x_i\}_{i=0}^{k-1}, \{\hat{q}_i \cdot \hat{q}_i^{-1}\}_{i=0}^{k-1} \rangle \pmod{Q} \tag{A.2}$$

A.2 Number Theoretic Transform

- Turns a convolution into a nega/cyclic
- Same as FFT but over different ring

Appendix B

Noise Asymmetry in RGSW Products

Theorem 2.29 notes an asymmetry . Here is the full derivation

$$
\begin{gathered}
x \cdot C = G^{-1}(x)(Z + \mu G) \\
G^{-1}(x) \cdot Z + \mu(G^{-1}(x) \cdot G) \\
G^{-1}(x)Z + \mu(x + \epsilon_g)
\end{gathered}
$$

where $\|G^{-1}\|_\infty \leq \beta$ is inhertied

$$\text{Using (2.22): } G^{-1}(x)Z = [D(x_0)|D(x_1)] \begin{pmatrix} \text{RLWE}(0) \\ \vdots \\ \text{RLWE}(0) \end{pmatrix}$$

$$\text{Let } D_i = G^{-1}(x)[i] \text{ and } Z_i = (a_i \cdot s + \epsilon_i, a_i)$$

$$\implies G^{-1}(x) \cdot Z = \left(\sum_{i=0}^{2\ell-1} a_i D_i s + \epsilon_i D_i, \quad \sum_{i=0}^{2\ell-1} a_i D_i \right)$$

$$\left(\left[\sum_{i=0}^{2\ell-1} a_i D_i \right] s + \sum_{i=0}^{2\ell-1} \epsilon_i D_i, \quad \sum_{i=0}^{2\ell-1} a_i D_i \right)$$

$$\text{Let } a = \sum_{i=0}^{2\ell-1} a_i D_i \text{ and } \epsilon = \sum_{i=0}^{2\ell-1} \epsilon_i, \text{ and recall } \|D(x_i)\|_\infty \leq \beta$$

$$(a \cdot s + \sum_{i=0}^{2\ell-1} \epsilon_i D_i, a) \leq (a \cdot s + \sum_{i=0}^{2\ell-1} \epsilon_i \beta, a)$$

$$\therefore G^{-1}(x) \cdot Z \leq (a \cdot s + \beta\epsilon, a)$$

So clearly, the error sources are:
– $\mu\epsilon_g$ which is 0 if the gadget is exact
– $\beta \sum e_z$ the sum of

$xxxx$